

UML for Database Design

Exam cheatsheet for conceptual modeling

In this course, a UML class diagram is not used to design program classes. It is used as a conceptual database schema: classes are entity types, attributes are stored data, associations are relationships, multiplicities are cardinality constraints, and comments are where the hard business rules go.

Core Notation

Notation	Database meaning	Exam use
Class	Entity type or concept in the domain.	Usually becomes a relation after mapping.
attribute: Type	Stored property of the entity or relationship.	Keep behavior and methods out of database UML unless explicitly requested.
attribute {id}	Identifier of an object in that class.	If the statement gives no reliable identifier, add a synthetic one such as camperId {id}.
/attribute	Derived attribute computed from other data.	Avoid storing it unless the statement asks for it.
<<enumeration>>	Finite domain of allowed values.	Use for values such as tent, caravan, house.
Association line	Relationship between entity types.	Name the association only when it clarifies the domain.
Role name	Meaning of one class from the point of view of the other.	Useful when the same two classes are connected more than once.
Association class	A relationship that has attributes of its own.	Use for Rental(price, arrivalDate, departureDate), not for Camper or Place.
*--	Composition: part belongs to a whole and is identified in that context.	Good for Camping composed of Zone, and Zone composed of Place.
< --	Inheritance or specialization.	Use only when the text says one concept is a subtype of another.
{constraint}	Comment or invariant not represented exactly by boxes and lines.	Use for temporal overlap, at-most constraints, and cross-object rules.

Rendering note: Pintora's class-member grammar rejects the course-specific {id} suffix inside class boxes. The diagrams therefore keep member syntax grammar-safe, and the tables state exactly which attributes must be marked {id} when copying the answer by hand.

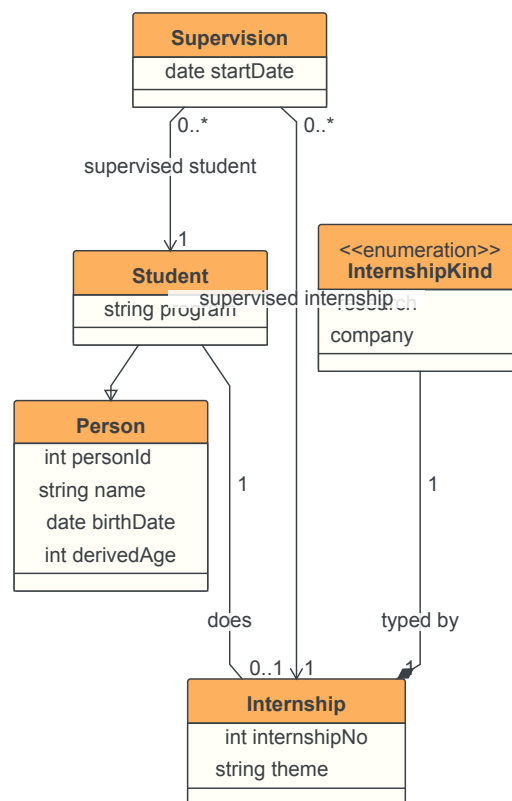


Figure 1: Small notation example: identifiers, enumeration, association, association entity, composition, and inheritance.

Multiplicities

Multiplicity is written at an association end. Read it as the number of objects at that end that may be linked to one object at the opposite end.

Multiplicity	Meaning	Typical wording
1	Exactly one.	"Each zone belongs to a camping."

Multiplicity	Meaning	Typical wording
0..1	Zero or one; optional but not many.	"A car may have an owner."
0..*	Zero, one, or many.	"A camping can have several facilities."
1..*	One or more.	"A camping consists of several zones."
0..2	At most two.	"A client can rent at most 2 places."

For an association A "1" -- "0..*" B, each B is linked to exactly one A, and each A can be linked to zero or more Bs. On an exam, write the multiplicity that follows from the sentence, then add a comment if the rule is temporal or depends on dates.

From Text To Diagram

Move	What to do	Camping example
Find stable nouns.	Turn domain objects into classes. Ignore implementation words.	Camping, Zone, Place, Facility, Camper.
Find descriptive data.	Put scalar data inside the class that owns it.	surface belongs to Place; bankAccount belongs to Camper.
Find relationship data.	If data describes a relationship, create an association class or explicit association entity.	price, arrivalDate, departureDate belong to Rental.
Find identifiers.	Mark natural identifiers with {id} or add a synthetic identifier when the statement gives none.	camperId {id} is safer than using name.
Find cardinalities.	Translate "one", "several", "at most", "can" into multiplicities.	Camping 1 to Zone 1..*; Zone 1 to Place 1..*.
Find hard constraints.	Write comments for constraints that are not simple multiplicities.	No overlapping rentals for the same place.

Worked Exam Example: Camping

The following is an exam-ready answer for the camping question. It chooses synthetic identifiers where the statement does not give globally unique attributes. It represents Rental as an explicit association entity, which is the database-friendly equivalent of an association class between Camper and Place.

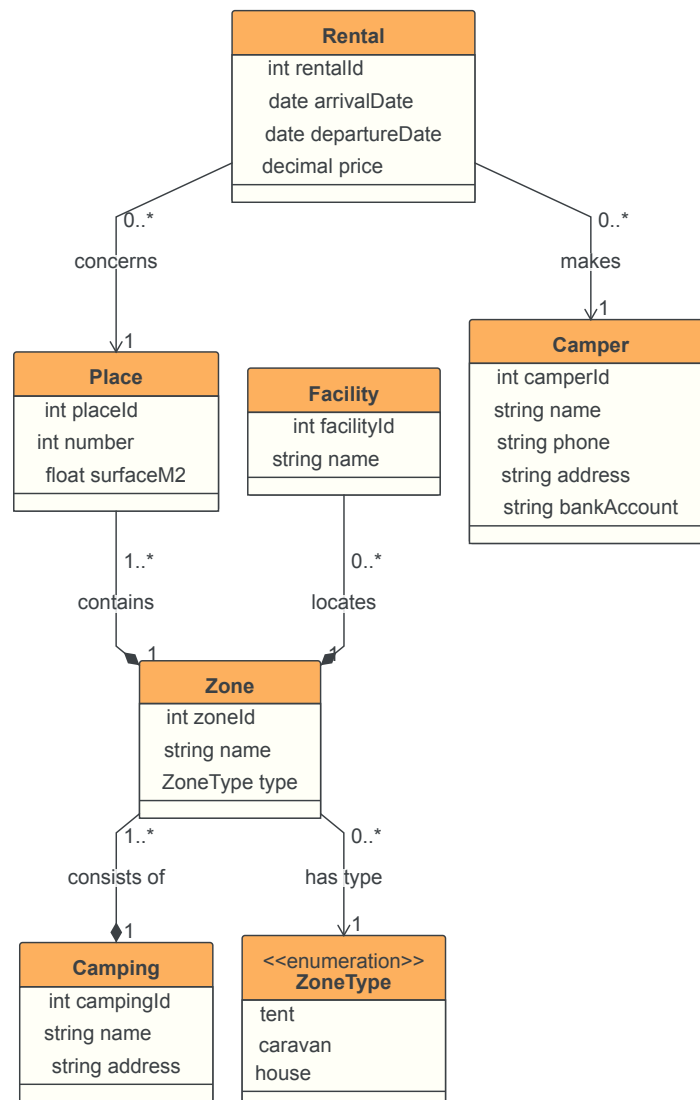


Figure 2: Conceptual UML design for the camping-site exam example.

The diagram intentionally keeps rentals as history. Therefore Camper and Place connect to Rental with 0..*, while the active-state limits are written as comments. If the exam statement is interpreted as only the current state and no rental history, the same domain could be simplified to an association class Rental on Camper "0..2" -- "0..1" Place.

Class	Identifier to mark	Why this is exam-safe
Camping	campingId {id}	The statement gives a name and address but does not state that names are unique.
Zone	zoneId {id}	Zone names may be unique only inside a camping, so a synthetic identifier avoids ambiguity.
Place	placeId {id}	The statement gives a place number, but not whether it is globally unique.
Facility	facilityId {id}	Facility names such as restaurant or bar can repeat across campings.
Camper	camperId {id}	Name, phone, address, and bank account are data; none is explicitly declared as a stable public identifier.
Rental	rentalId {id}	A synthetic identifier is simple; an alternative is a composite identifier such as (placeId, arrivalDate).

Exam comments to write	Reason
{Zone.type in {tent, caravan, house}}	The text gives a finite domain, so this can be represented by the ZoneType enumeration or by a UML comment.
{arrivalDate < departureDate}	The dates are attributes of Rental; the order between them is a value constraint.
{For a fixed Place, rental date intervals must not overlap}	A place can only be rented by one camper at a time. This is temporal and cannot be shown by simple multiplicity when history is stored.
{For a fixed Camper, at most two rental intervals may overlap}	A camper can rent at most two places at the same time. This is also temporal if rentals are historical.
{Place.number is unique inside its Zone or Camping if placeId is not used}	The statement says each place has a number but does not clearly say whether the number is globally unique.

Why The Attributes Go There

Data	Correct owner	Reason
surfaceM2	Place	The surface describes the physical place, independently of who rents it.
type	Zone	The type determines who can camp in all places of the zone.
name of a facility	Facility	A facility is a thing located in a zone.
price	Rental	The price is not fixed for the camper or the place; it belongs to a particular rental event.
arrivalDate, departureDate	Rental	Dates describe the period of the relationship between one camper and one place.
bankAccount	Camper	The account is part of client payment information, not the rental itself unless the statement says it changes per rental.

Relational Consequence

The UML answer is enough for the design exercise, but knowing the relational consequence helps check whether the model is sensible. A clean mapping would produce relations shaped like this:

```

Camping(campingId, name, address)
Zone(zoneId, campingId, name, type)
Place(placeId, zoneId, number, surfaceM2)
Facility(facilityId, zoneId, name)
Camper(camperId, name, phone, address, bankAccount)
Rental(rentalId, camperId, placeId, arrivalDate, departureDate, price)

```

The foreign keys are Zone.campingId -> Camping.campingId, Place.zoneId -> Zone.zoneId, Facility.zoneId -> Zone.zoneId, Rental.camperId -> Camper.camperId, and Rental.placeId -> Place.placeId. The temporal constraints about overlapping rentals are not ordinary foreign keys and must be stated separately in UML comments.

Mapping Patterns To Remember

UML pattern	Relational effect	Exam warning
1 to 0..*	Foreign key on the many side.	Do not create an extra relation unless the relationship has attributes or optionality makes nulls undesirable.
0..* to 0..*	New relation containing both identifiers.	This is where relationship attributes also go.
Association class	New relation containing participant identifiers plus association attributes.	Never put relationship attributes in only one participating class.

UML pattern	Relational effect	Exam warning
Composition	Part relation references the whole; the whole identifier may be part of the part key.	Use when the part is identified only inside the whole.
Inheritance	Possible mappings are reference, duplication, or one unified relation.	Mention disjoint, overlapping, full, or partial constraints if the statement gives them.

Fast Quality Check

Question before submitting	What a good answer does
Does every class have an identifier?	Yes, or there is a clear comment explaining composite/contextual identification.
Did I put dates and price on the relationship?	Yes, Rental owns them because they describe the act of renting.
Did I show that facilities are in zones?	Yes, Facility is linked to exactly one Zone.
Did I confuse current state with history?	No, history uses Rental 0..* plus overlap comments; current-state-only can use stricter direct multiplicities.
Did I write the hard constraints?	Yes, especially non-overlap, at most two active rentals, and date ordering.